Smart Dynamic Sampling Rate System

Adaptive Power-Aware Monitoring using FreeRTOS Non-Blocking Tasks

Institution: Padmabhooshan Vasantraodada Patil Institute of Technology (PVPIT), Budhgaon, Sangli

Project Track: Power-Optimized Embedded Systems & RTOS Concurrency

Document: Technical Specification & System Report

---

1. System Architecture Diagram

The system is architected to dynamically scale power consumption and message frequency based on asynchronous network events or autonomous health evaluations. Interval transitions are routed through FreeRTOS queues, guaranteeing that task parameter changes occur safely without blocking runtime loops or losing telemetry data.



2. FreeRTOS Task Structure Explanation

To implement an adaptive scheduling system without risking task lockups or starvation, all operational intervals rely on non-blocking FreeRTOS primitives. Tasks are split into explicit execution domains.

Task Priority and Interval Structure

Task Handle Name	Priority	Execution Strategy	Functional System Objective
vTaskSensorSample	High (4)	Dynamic Interval	Polls physical hardware metrics. Sleep durations adjust dynamically based on queue notifications.
vTaskVitalsEval	High (3)	Event-Driven	Evaluates patient data trends; calculates and overrides sampling frequencies in Auto mode.
vTaskMQTTComm	Medium (2)	200ms (Periodic)	Monitors network sockets, forwards vitals data streams, and catches manual dashboard overrides.
vTaskDisplayUI	Low (1)	300ms (Periodic)	Updates local OLED layouts, rendering active intervals, power statistics, and warning indicators.
vTaskWatchdog	Highest (5)	100ms (Periodic)	Monitors core tasks; automatically switches back to standard safety periods if an error is detected.

Inter-Task Data Communication Primitives

- **Sampling Rate Queue (`xSamplingQueue`):** A dedicated single-element thread-safe queue used to send new time interval tokens directly to the sensory loop task without interrupting current execution.
- **Vitals State Mutex:** Wraps around operational flags (such as Auto/Manual mode and current intervals) to ensure data consistency across multiple processing loops.

3. System Feed Mapping Table

Network communications use structured MQTT topics to isolate user adjustments from real-time performance logs and telemetry streams.

MQTT Topic Namespace	Data Format	Direction	Description / Operational Context
`/sampling/mode/set`	String (`"AUTO"` / `"MANUAL"`)	Subscribe	Configures system behavior to follow autonomous logic or dashboard manual slider overrides.
`/sampling/interval/set`	Integer (5 - 60)	Subscribe	Sets the manual target tracking frequency in seconds (active in Manual mode).
`/sampling/interval/current`	Integer (5 - 60)	Publish	Reports the active sampling interval currently being executed by the edge controller.
`/sampling/power/metrics`	JSON String	Publish	Transmits power efficiency parameters including estimated current draw reduction and uptime statistics.

4. Remote UI Control Dashboard Design

The system control panel includes functional mode toggles, explicit adjustments, and dynamic system tracking charts.

*[**Power-Aware Sensor Management Console View**]*
• Mode Configuration Switch: Dynamic toggle switch selecting between Manual Control and Autonomous Adaptive Mode.
• Target Frequency Adjustment Slider: Linear input slider bounded from 5 seconds (High-Fidelity) to 60 seconds (Eco-Saving Mode).
• Optimization Log Window: Real-time telemetry chart mapping active sample rates alongside calculated battery savings.

5. Non-Blocking Delay & Queue Implementation

To keep the processor responsive and avoid wasteful spinning loops during long sleep periods (up to 60 seconds), the system utilizes non-blocking RTOS sleep functions:

```
vTaskDelay(pdMS_TO_TICKS(current_delay_ms));
```

This puts the sampling task into a suspended state, allowing the FreeRTOS kernel to free up processor cycles for lower-priority tasks like network management and display updates. When the interval needs to change, instead of using messy global variables, the system uses safe queues to pass new timing values:

```
// Non-Blocking Dynamic Sampling Loop Implementation
void vTaskSensorSample(void *pvParameters) {
    uint32_t active_interval_seconds = 10; // Default safety fallback baseline
    uint32_t received_token;

    for(;;) {
        // Check for new interval updates in the queue without blocking execution
        if (xQueueReceive(xSamplingQueue, &received_token, 0) == pdTRUE) {
            active_interval_seconds = received_token;
        }

        execute_sensor_read_sequence();

        // Pass control back to the kernel for the requested sleep duration
        vTaskDelay(pdMS_TO_TICKS(active_interval_seconds * 1000));
    }
}
```

6. Patient-Condition Adaptive Control Logic

When running in **Automatic Mode**, the system dynamically scales tracking resolution based on patient health indicators:

- **High-Fidelity Emergency State (5s Interval):** If vitals data drops below or spikes past predefined clinical boundaries (e.g., Heart Rate > 120 BPM or SpO2 < 93%), the system immediately switches to a 5-second sampling rate to capture fine-grained data for emergency triage.
- **Standard Baseline State (15s Interval):** When vitals stabilize within a normal range, the system scales back to a moderate 15-second tracking rate to conserve system resources.
- **Eco-Efficiency State (60s Interval):** If patient metrics remain completely stable during extended sleep or rest periods, the system reduces sampling to once every 60 seconds, maximizing battery life.

7. Battery Optimization Analytics

The system achieves significant energy savings by varying device sleep windows based on operational demands. Power savings are evaluated using a standardized calculation matrix:

$$P_{total} = \left(\frac{T_{active}}{T_{interval}} \times P_{active} \right) + \left(\left(1 - \frac{T_{active}}{T_{interval}} \right) \times P_{sleep} \right)$$

Where $P_{active} = 120 \text{ mA}$ (radio and sensors active) and $P_{sleep} = 15 \text{ mA}$ (radio idle, processor suspended). The active measurement window takes $T_{active} = 400 \text{ ms}$.

- At a high-frequency **5-second interval**, the device spends 8% of its time active, resulting in an average current draw of **23.4 mA**.

- At an optimized **60-second interval**, active time drops to just 0.66%, reducing average current draw to **$15.7 ext{ mA}$**.

This dynamic adjustment yields an estimated **32.9% reduction in overall energy consumption** during stable tracking periods, significantly extending battery life for mobile medical devices.

8. Safety Bounds & Fault Fallback Routines

To ensure patient safety is never compromised for power savings, the firmware incorporates strict operational guardrails:

- **Enforced Ceiling Limits:** Hardcoded checks prevent the manual slider or auto logic from exceeding a 60-second delay, guaranteeing patient data is updated at least once a minute.
- **Asymmetric Fast-Attack Logic:** The system is designed to scale up frequency instantly but delay scaling down. When an anomaly is detected, the sampling rate jumps to 5 seconds within one execution cycle. However, when vitals return to normal, the system requires 5 minutes of continuous stability before scaling down to eco mode, ensuring fleeting issues are not missed.
- **Watchdog Reset Strategy:** If a task exception or queue error causes an unexpected crash, the `vTaskWatchdog` catches the fault and automatically forces the system back to the safe, high-frequency 5-second sampling rate.